

report

May 22, 2025

1 Lista 2: Symulacja Clobber

1.1 Sztuczna Inteligencja 2025

Data: 22 maja 2025 **Autor:** Wojciech Krzos 276264

Niniejszy raport przedstawia przegląd i analizę symulacji stworzonej na potrzeby Listy 2. Symulacja implementuje algorytmy minimax i alpha-beta pruning do gry w Clobber. Wy tłumaczenie zasad zadania oraz gry znajduje się w treści Listy 2.

1.2 1. Struktura Projektu

Projekt jest zorganizowany w modularnej strukturze z wyraźnym rozdzieleniem odpowiedzialności:

lab2/

```
main.py          # Punkt wejścia do uruchamiania symulacji
utils.py         # Funkcje jak Timer
game/           # Komponenty specyficzne dla gry
  agent.py       # Implementacja agenta AI
  alphabeta.py   # Algorytm przycinania alpha-beta
  board.py       # Reprezentacja planszy i logika gry
  heuristics.py  # Funkcje oceny
  minimax.py     # Implementacja algorytmu minimax
```

Poniżej analizowane są kolejne komponenty oraz użyte biblioteki.

1.2.1 1.0 Biblioteki i zależności

Używane biblioteki standardowe Pythona to:

- **sys:** Do obsługi wejścia/wyjścia i argumentów systemowych
- **argparse:** Do parsowania argumentów wiersza poleceń
- **time:** Do mierzenia czasu wykonania funkcji (użyty w Timer)
- **os, select:** Do opcjonalnego odczytu danych z stdin

Projekt jest lekki i łatwy do uruchomienia w dowolnym środowisku z Pythonem, bez konieczności instalowania dodatkowych pakietów. Algorytmy implementowane są od podstaw.

1.2.2 1.1 Reprezentacja Planszy (board.py)

Klasa Board zarządza stanem gry, śledzi pionki i egzekwuje zasady gry:

```

[ ]: # Board class implements the game logic
class Board:
    def __init__(self, grid):
        self.grid = [row[:] for row in grid] # Deep copy of the grid
        self.rows = len(grid)
        self.cols = len(grid[0])

    def clone(self):
        return Board(self.grid)

    def get_moves(self, player):
        # Find all possible moves for a player
        enemy = 'B' if player=='W' else 'W'
        moves = []
        for i in range(self.rows):
            for j in range(self.cols):
                if self.grid[i][j]==player:
                    # Check orthogonal directions for captures
                    for di,dj in [(1,0),(-1,0),(0,1),(0,-1)]:
                        ni,nj = i+di, j+dj
                        if 0<=ni<self.rows and 0<=nj<self.cols and self.
↪grid[ni][nj]==enemy:
                            moves.append(((i,j),(ni,nj)))
        return moves

    def apply_move(self, move):
        # Apply a move to the board
        (i,j),(ni,nj) = move
        p = self.grid[i][j]
        self.grid[i][j] = '_' # Empty the source cell
        self.grid[ni][nj] = p # Move piece to target cell

    def is_terminal(self, player):
        # Check if game is over for a player (no more moves)
        return not self.get_moves(player)

    def winner(self, last_player):
        # The last player able to move wins
        return last_player

    def __str__(self):
        return '\n'.join(' '.join(row) for row in self.grid)

```

1.2.3 1.2 Funkcje Heurystyczne (heuristics.py)

```
[ ]: # Heuristic functions to evaluate board positions
def h_count(board, player):
    """Evaluates position based on piece count difference"""
    enemy = 'B' if player=='W' else 'W'
    pc = sum(row.count(player) for row in board.grid) # Player's pieces
    ec = sum(row.count(enemy) for row in board.grid) # Enemy's pieces
    return pc-ec # Difference favors the player

def h_mobility(board, player):
    """Evaluates position based on move advantage"""
    enemy = 'B' if player=='W' else 'W'
    return len(board.get_moves(player))-len(board.get_moves(enemy))

def h_combo(board, player):
    """Combined heuristic using both piece count and mobility"""
    return h_count(board,player)*2 + h_mobility(board,player)

# Dictionary of available heuristics
HEURISTICS = {'count':h_count, 'mobility':h_mobility, 'combo':h_combo}
```

1.2.4 1.3 Algorytmy Wyszukiwania

1.3.1 Minimax (minimax.py)

```
[ ]: import time
from utils import Timer

def minimax(board, player, depth, heur):
    nodes = 0 # Keep track of explored nodes for performance analysis

    @Timer
    def _minimax(node, pl, d):
        nonlocal nodes
        nodes += 1

        # Terminal node evaluation
        if d == 0 or node.is_terminal(pl):
            return heur(node, pl), None

        # Initialize best value based on player (max or min)
        best_val, best_mv = (float('-inf'), None) if pl == player else
        ↪(float('inf'), None)

        # Explore all possible moves
        for mv in node.get_moves(pl):
            nb = node.clone()
```

```

        nb.apply_move(mv)
        val, _ = _minimax(nb, 'B' if pl == 'W' else 'W', d-1)

        # Update best move if this move is better
        if (pl == player and val > best_val) or (pl != player and val <
↪best_val):
            best_val, best_mv = val, mv

    return best_val, best_mv

# Call the search with timing
(result, move), dur = _minimax(board, player, depth)
return move, nodes, dur

```

1.3.2 Przycinanie Alpha-Beta (alphabeta.py)

```

[ ]: from utils import Timer

def alphabeta(board, player, depth, heur):
    nodes = 0

    def _ab(node, pl, d, a, b):
        nonlocal nodes
        nodes += 1

        # Terminal node evaluation
        if d == 0 or node.is_terminal(pl):
            return heur(node, pl), None

        best_mv = None

        # Maximizing player
        if pl == player:
            val = float('-inf')
            for mv in node.get_moves(pl):
                nb = node.clone()
                nb.apply_move(mv)
                v, _ = _ab(nb, 'B' if pl == 'W' else 'W', d-1, a, b)
                if v > val:
                    val, best_mv = v, mv
                a = max(a, val)
                if a >= b: # Beta cutoff
                    break
            return val, best_mv

        # Minimizing player
        else:
            val = float('inf')

```

```

        for mv in node.get_moves(pl):
            nb = node.clone()
            nb.apply_move(mv)
            v, _ = _ab(nb, 'B' if pl == 'W' else 'W', d-1, a, b)
            if v < val:
                val, best_mv = v, mv
            b = min(b, val)
            if b <= a: # Alpha cutoff
                break
        return val, best_mv

@Timer
def wrapper():
    val, mv = _ab(board, player, depth, float('-inf'), float('inf'))
    return val, mv

# Call the search with timing
(val, mv), dur = wrapper()
return mv, nodes, dur

```

1.2.5 1.4 Implementacja Agenta (agent.py)

Klasa Agent obsługuje procesy podejmowania decyzji:

```

[ ]: class Agent:
    def __init__(self, name, method, heur_name, depth):
        self.name = name
        self.method = method # 'minimax' or 'alphabeta'
        self.heur = HEURISTICS[heur_name] # Selected heuristic function
        self.depth = depth # Search depth

    def move(self, board, player):
        # Choose the search algorithm based on agent configuration
        if self.method == 'minimax':
            return minimax(board, player, self.depth, self.heur)
        return alphabeta(board, player, self.depth, self.heur)

```

1.2.6 1.5 Narzędzie Timera (utils.py)

Dekorator (wspomniany również powyżej, w opisie bibliotek) do mierzenia czasu wykonania funkcji:

```

[ ]: import time

def Timer(f):
    def inner(*a, **k):
        t0 = time.time()
        res = f(*a, **k)
        return res, (time.time() - t0)

```

```
return inner
```

1.3 2. Opis Gry Clobber

Stwierdzono następujące cechy gry na podstawie których zaimplementowano symulację:

- Rozgrywana jest na prostokątnej siatce (domyślnie 5x6) z czarnymi ('B') i białymi ('W') pionkami
- Gracze na zmianę poruszają swoimi pionkami, aby zbijać pionki przeciwnika
- Pionek może zbić sąsiedni pionek przeciwnika, przesuając się na jego pozycję
- Zbijanie jest dozwolone tylko w kierunkach ortogonalnych (góra, dół, lewo, prawo)*
- Gra kończy się, gdy gracz nie ma dostępnych legalnych ruchów
- Wygrywa gracz, który jako ostatni był w stanie wykonać ruch

Gracze AI używają algorytmów minimax lub alpha-beta pruning z różnymi funkcjami oceny, aby określić najlepsze ruchy.

*W niektórych wersjach gry dopuszczalne są również ruchy na skos.

1.4 3. Użycie z Linii Poleceń

Symulacja jest uruchamiana za pomocą skryptu `main.py` z różnymi argumentami linii poleceń do konfiguracji gry:

```
[ ]: # Basic command structure (not to be executed in notebook)
'''
python main.py --heur1 <heuristic1> --heur2 <heuristic2> --depth <search_depth>
    ↪ [options]
'''
```

1.4.1 Argumenty Linii Poleceń

Argument	Opis	Wartości
--heur1	Heurystyka dla gracza 1	count, mobility, combo
--heur2	Heurystyka dla gracza 2	count, mobility, combo
--depth	Głębokość wyszukiwania dla AI	Liczba całkowita (np. 3, 4, 5)
--method	Algorytm wyszukiwania	minimax lub alphabeta (domyślnie)
--start	Rozpoczynający gracz	B lub W (domyślnie: B)
--debug	Włącz wyjście debugowania	Flaga (nie wymaga wartości)
--history	Wyświetl historię planszy	Flaga (nie wymaga wartości)
--file	Plik wejściowy z konfiguracją planszy	Ścieżka do pliku

1.5 4. Przykłady Użycia

1.5.1 Przykład 1: Podstawowy Minimax z Różnymi Heurystykami

Ten przykład uruchamia grę z algorytmem minimax, gdzie obaj gracze używają różnych strategii oceny: Gracz 1 używa liczby pionków, podczas gdy Gracz 2 używa mobilności.

```
[ ]: # python main.py --heur1 count --heur2 mobility --depth 3 --method minimax
- - - - -
- _ B _ _ W
_ B _ _ W _
B _ B _ _ W
B B _ W W W
18 rounds, winner B
364222
3.4785
```

1.5.2 Przykład 2: Przycinanie Alpha-Beta z Tą Samą Heurystyką

Ten przykład demonstruje grę, w której obaj gracze używają przycinania alpha-beta z połączoną heurystyką do oceny, ale na różnych głębokościach wyszukiwania.

```
[ ]: # python main.py --heur1 combo --heur2 combo --depth 4 --method alphabeta
    ↪--start W --debug
=== INITIAL BOARD ===
B W B W B W
W B W B W B
B W B W B W
W B W B W B
B W B W B W
=====
Starting game with player W
Round 1: Player W thinking...
Available moves: [((0, 1), (1, 1)), ((0, 1), (0, 2)), ((0, 1), (0, 0)), ((0, ↪
    ↪3), (1, 3)), ((0, 3), (0, 4)), ((0, 3), (0, 2)), ((0, 5), (1, 5)), ((0, 5), ↪
    ↪(0, 4)), ((1, 0), (2, 0)), ((1, 0), (0, 0)), ((1, 0), (1, 1)), ((1, 2), (2, ↪
    ↪2)), ((1, 2), (0, 2)), ((1, 2), (1, 3)), ((1, 2), (1, 1)), ((1, 4), (2, 4)), ↪
    ↪((1, 4), (0, 4)), ((1, 4), (1, 5)), ((1, 4), (1, 3)), ((2, 1), (3, 1)), ((2, ↪
    ↪1), (1, 1)), ((2, 1), (2, 2)), ((2, 1), (2, 0)), ((2, 3), (3, 3)), ((2, 3), ↪
    ↪(1, 3)), ((2, 3), (2, 4)), ((2, 3), (2, 2)), ((2, 5), (3, 5)), ((2, 5), (1, ↪
    ↪5)), ((2, 5), (2, 4)), ((3, 0), (4, 0)), ((3, 0), (2, 0)), ((3, 0), (3, 1)), ↪
    ↪((3, 2), (4, 2)), ((3, 2), (2, 2)), ((3, 2), (3, 3)), ((3, 2), (3, 1)), ((3, ↪
    ↪4), (4, 4)), ((3, 4), (2, 4)), ((3, 4), (3, 5)), ((3, 4), (3, 3)), ((4, 1), ↪
    ↪(3, 1)), ((4, 1), (4, 2)), ((4, 1), (4, 0)), ((4, 3), (3, 3)), ((4, 3), (4, ↪
    ↪4)), ((4, 3), (4, 2)), ((4, 5), (3, 5)), ((4, 5), (4, 4))]
```

```
Player W chose move: ((0, 1), (1, 1))
Nodes explored: 5442, Time taken: 0.1154s
Board after move:
B _ B W B W
```

```

W W W B W B
B W B W B W
W B W B W B
B W B W B W

```

Round 2: Player B thinking...

Available moves: [((0, 0), (1, 0)), ((0, 2), (1, 2)), ((0, 2), (0, 3)), ((0, 4), (1, 4)), ((0, 4), (0, 5)), ((0, 4), (0, 3)), ((1, 3), (2, 3)), ((1, 3), (0, 3)), ((1, 3), (1, 4)), ((1, 3), (1, 2)), ((1, 5), (2, 5)), ((1, 5), (0, 5)), ((1, 5), (1, 4)), ((2, 0), (3, 0)), ((2, 0), (1, 0)), ((2, 0), (2, 1)), ((2, 2), (3, 2)), ((2, 2), (1, 2)), ((2, 2), (2, 3)), ((2, 2), (2, 1)), ((2, 4), (3, 4)), ((2, 4), (1, 4)), ((2, 4), (2, 5)), ((2, 4), (2, 3)), ((3, 1), (4, 1)), ((3, 1), (2, 1)), ((3, 1), (3, 2)), ((3, 1), (3, 0)), ((3, 3), (4, 3)), ((3, 3), (2, 3)), ((3, 3), (3, 4)), ((3, 3), (3, 2)), ((3, 5), (4, 5)), ((3, 5), (2, 5)), ((3, 5), (3, 4)), ((4, 0), (3, 0)), ((4, 0), (4, 1)), ((4, 2), (3, 2)), ((4, 2), (4, 3)), ((4, 2), (4, 1)), ((4, 4), (3, 4)), ((4, 4), (4, 5)), ((4, 4), (4, 3))]

Player B chose move: ((0, 0), (1, 0))

Nodes explored: 4375, Time taken: 0.0896s

Board after move:

```

_ _ B W B W
B W W B W B
B W B W B W
W B W B W B
B W B W B W

```

Round 3: Player W thinking...

Available moves: [((0, 3), (1, 3)), ((0, 3), (0, 4)), ((0, 3), (0, 2)), ((0, 5), (1, 5)), ((0, 5), (0, 4)), ((1, 1), (1, 0)), ((1, 2), (2, 2)), ((1, 2), (0, 2)), ((1, 2), (1, 3)), ((1, 4), (2, 4)), ((1, 4), (0, 4)), ((1, 4), (1, 5)), ((1, 4), (1, 3)), ((2, 1), (3, 1)), ((2, 1), (2, 2)), ((2, 1), (2, 0)), ((2, 3), (3, 3)), ((2, 3), (1, 3)), ((2, 3), (2, 4)), ((2, 3), (2, 2)), ((2, 5), (3, 5)), ((2, 5), (1, 5)), ((2, 5), (2, 4)), ((3, 0), (4, 0)), ((3, 0), (2, 0)), ((3, 0), (3, 1)), ((3, 2), (4, 2)), ((3, 2), (2, 2)), ((3, 2), (3, 3)), ((3, 2), (3, 1)), ((3, 4), (4, 4)), ((3, 4), (2, 4)), ((3, 4), (3, 5)), ((3, 4), (3, 3)), ((4, 1), (3, 1)), ((4, 1), (4, 2)), ((4, 1), (4, 0)), ((4, 3), (3, 3)), ((4, 3), (4, 4)), ((4, 3), (4, 2)), ((4, 5), (3, 5)), ((4, 5), (4, 4))]

Player W chose move: ((0, 3), (1, 3))

Nodes explored: 3996, Time taken: 0.0790s

Board after move:

```

_ _ B _ B W
B W W W W B
B W B W B W
W B W B W B
B W B W B W

```


Round 4: Player B thinking...

Available moves: [((0, 2), (1, 2)), ((0, 4), (1, 4)), ((0, 4), (0, 5)), ((1, 0), (1, 1)), ((1, 5), (2, 5)), ((1, 5), (0, 5)), ((1, 5), (1, 4)), ((2, 0), (3, 0)), ((2, 0), (2, 1)), ((2, 2), (3, 2)), ((2, 2), (1, 2)), ((2, 2), (2, 3)), ((2, 2), (2, 1)), ((2, 4), (3, 4)), ((2, 4), (1, 4)), ((2, 4), (2, 5)), ((2, 4), (2, 3)), ((3, 1), (4, 1)), ((3, 1), (2, 1)), ((3, 1), (3, 2)), ((3, 1), (3, 0)), ((3, 3), (4, 3)), ((3, 3), (2, 3)), ((3, 3), (3, 4)), ((3, 3), (3, 2)), ((3, 5), (4, 5)), ((3, 5), (2, 5)), ((3, 5), (3, 4)), ((4, 0), (3, 0)), ((4, 0), (4, 1)), ((4, 2), (3, 2)), ((4, 2), (4, 3)), ((4, 2), (4, 1)), ((4, 4), (3, 4)), ((4, 4), (4, 5)), ((4, 4), (4, 3))]

Player B chose move: ((0, 2), (1, 2))

Nodes explored: 3182, Time taken: 0.0641s

Board after move:

```
- - - - B W
B W B W W B
B W B W B W
W B W B W B
B W B W B W
```

Round 5: Player W thinking...

Available moves: [((0, 5), (1, 5)), ((0, 5), (0, 4)), ((1, 1), (1, 2)), ((1, 1), (1, 0)), ((1, 3), (1, 2)), ((1, 4), (2, 4)), ((1, 4), (0, 4)), ((1, 4), (1, 5)), ((2, 1), (3, 1)), ((2, 1), (2, 2)), ((2, 1), (2, 0)), ((2, 3), (3, 3)), ((2, 3), (2, 4)), ((2, 3), (2, 2)), ((2, 5), (3, 5)), ((2, 5), (1, 5)), ((2, 5), (2, 4)), ((3, 0), (4, 0)), ((3, 0), (2, 0)), ((3, 0), (3, 1)), ((3, 2), (4, 2)), ((3, 2), (2, 2)), ((3, 2), (3, 3)), ((3, 2), (3, 1)), ((3, 4), (4, 4)), ((3, 4), (2, 4)), ((3, 4), (3, 5)), ((3, 4), (3, 3)), ((4, 1), (3, 1)), ((4, 1), (4, 2)), ((4, 1), (4, 0)), ((4, 3), (3, 3)), ((4, 3), (4, 4)), ((4, 3), (4, 2)), ((4, 5), (3, 5)), ((4, 5), (4, 4))]

Player W chose move: ((0, 5), (1, 5))

Nodes explored: 3017, Time taken: 0.0581s

Board after move:

```
- - - - B -
B W B W W W
B W B W B W
W B W B W B
B W B W B W
```

Round 6: Player B thinking...

Available moves: [((0, 4), (1, 4)), ((1, 0), (1, 1)), ((1, 2), (1, 3)), ((1, 2), (1, 1)), ((2, 0), (3, 0)), ((2, 0), (2, 1)), ((2, 2), (3, 2)), ((2, 2), (2, 3)), ((2, 2), (2, 1)), ((2, 4), (3, 4)), ((2, 4), (1, 4)), ((2, 4), (2, 5)), ((2, 4), (2, 3)), ((3, 1), (4, 1)), ((3, 1), (2, 1)), ((3, 1), (3, 2)), ((3, 1), (3, 0)), ((3, 3), (4, 3)), ((3, 3), (2, 3)), ((3, 3), (3, 4)), ((3, 3), (3, 2)), ((3, 5), (4, 5)), ((3, 5), (2, 5)), ((3, 5), (3, 4)), ((4, 0), (3, 0)), ((4, 0), (4, 1)), ((4, 2), (3, 2)), ((4, 2), (4, 3)), ((4, 2), (4, 1)), ((4, 4), (3, 4)), ((4, 4), (4, 5)), ((4, 4), (4, 3))]

Player B chose move: ((0, 4), (1, 4))
Nodes explored: 2613, Time taken: 0.0474s
Board after move:

```
- - - - -
B W B W B W
B W B W B W
W B W B W B
B W B W B W
```

Round 7: Player W thinking...

Available moves: [((1, 1), (1, 2)), ((1, 1), (1, 0)), ((1, 3), (1, 4)), ((1, 1), (1, 5)), ((1, 2), (1, 4)), ((1, 5), (1, 4)), ((2, 1), (3, 1)), ((2, 1), (2, 2)), ((2, 1), (2, 0)), ((2, 3), (3, 3)), ((2, 3), (2, 4)), ((2, 3), (2, 2)), ((2, 5), (3, 5)), ((2, 5), (2, 4)), ((3, 0), (4, 0)), ((3, 0), (2, 0)), ((3, 0), (3, 1)), ((3, 2), (4, 2)), ((3, 2), (2, 2)), ((3, 2), (3, 3)), ((3, 2), (3, 1)), ((3, 4), (4, 4)), ((3, 4), (2, 4)), ((3, 4), (3, 5)), ((3, 4), (3, 3)), ((4, 1), (3, 1)), ((4, 1), (4, 2)), ((4, 1), (4, 0)), ((4, 3), (3, 3)), ((4, 3), (4, 4)), ((4, 3), (4, 2)), ((4, 5), (3, 5)), ((4, 5), (4, 4))]

Player W chose move: ((1, 1), (1, 2))
Nodes explored: 2480, Time taken: 0.0458s
Board after move:

```
- - - - -
B _ W W B W
B W B W B W
W B W B W B
B W B W B W
```

Round 8: Player B thinking...

Available moves: [((1, 4), (1, 5)), ((1, 4), (1, 3)), ((2, 0), (3, 0)), ((2, 0), (2, 1)), ((2, 2), (3, 2)), ((2, 2), (1, 2)), ((2, 2), (2, 3)), ((2, 2), (2, 1)), ((2, 4), (3, 4)), ((2, 4), (2, 5)), ((2, 4), (2, 3)), ((3, 1), (4, 1)), ((3, 1), (2, 1)), ((3, 1), (3, 2)), ((3, 1), (3, 0)), ((3, 3), (4, 3)), ((3, 3), (2, 3)), ((3, 3), (3, 4)), ((3, 3), (3, 2)), ((3, 5), (4, 5)), ((3, 5), (2, 5)), ((3, 5), (3, 4)), ((4, 0), (3, 0)), ((4, 0), (4, 1)), ((4, 2), (3, 2)), ((4, 2), (4, 3)), ((4, 2), (4, 1)), ((4, 4), (3, 4)), ((4, 4), (4, 5)), ((4, 4), (4, 3))]

Player B chose move: ((1, 4), (1, 5))
Nodes explored: 2048, Time taken: 0.0350s
Board after move:

```
- - - - -
B _ W W _ B
B W B W B W
W B W B W B
B W B W B W
```

Round 9: Player W thinking...

Available moves: [((1, 2), (2, 2)), ((2, 1), (3, 1)), ((2, 1), (2, 2)), ((2, 1), (2, 0)), ((2, 3), (3, 3)), ((2, 3), (2, 4)), ((2, 3), (2, 2)), ((2, 5), (3, 5)), ((2, 5), (1, 5)), ((2, 5), (2, 4)), ((3, 0), (4, 0)), ((3, 0), (2, 0)), ((3, 0), (3, 1)), ((3, 2), (4, 2)), ((3, 2), (2, 2)), ((3, 2), (3, 3)), ((3, 2), (3, 1)), ((3, 4), (4, 4)), ((3, 4), (2, 4)), ((3, 4), (3, 5)), ((3, 4), (3, 3)), ((4, 1), (3, 1)), ((4, 1), (4, 2)), ((4, 1), (4, 0)), ((4, 3), (3, 3)), ((4, 3), (4, 4)), ((4, 3), (4, 2)), ((4, 5), (3, 5)), ((4, 5), (4, 4))]

Player W chose move: ((1, 2), (2, 2))

Nodes explored: 1759, Time taken: 0.0301s

Board after move:

```

- - - - -
B _ _ W _ B
B W W W B W
W B W B W B
B W B W B W

```

Round 10: Player B thinking...

Available moves: [((1, 5), (2, 5)), ((2, 0), (3, 0)), ((2, 0), (2, 1)), ((2, 4), (3, 4)), ((2, 4), (2, 5)), ((2, 4), (2, 3)), ((3, 1), (4, 1)), ((3, 1), (2, 1)), ((3, 1), (3, 2)), ((3, 1), (3, 0)), ((3, 3), (4, 3)), ((3, 3), (2, 3)), ((3, 3), (3, 4)), ((3, 3), (3, 2)), ((3, 5), (4, 5)), ((3, 5), (2, 5)), ((3, 5), (3, 4)), ((4, 0), (3, 0)), ((4, 0), (4, 1)), ((4, 2), (3, 2)), ((4, 2), (4, 3)), ((4, 2), (4, 1)), ((4, 4), (3, 4)), ((4, 4), (4, 5)), ((4, 4), (4, 3))]

Player B chose move: ((1, 5), (2, 5))

Nodes explored: 1269, Time taken: 0.0203s

Board after move:

```

- - - - -
B _ _ W _ _
B W W W B B
W B W B W B
B W B W B W

```

Round 11: Player W thinking...

Available moves: [((2, 1), (3, 1)), ((2, 1), (2, 0)), ((2, 3), (3, 3)), ((2, 3), (2, 4)), ((3, 0), (4, 0)), ((3, 0), (2, 0)), ((3, 0), (3, 1)), ((3, 2), (4, 2)), ((3, 2), (3, 3)), ((3, 2), (3, 1)), ((3, 4), (4, 4)), ((3, 4), (2, 4)), ((3, 4), (3, 5)), ((3, 4), (3, 3)), ((4, 1), (3, 1)), ((4, 1), (4, 2)), ((4, 1), (4, 0)), ((4, 3), (3, 3)), ((4, 3), (4, 4)), ((4, 3), (4, 2)), ((4, 5), (3, 5)), ((4, 5), (4, 4))]

Player W chose move: ((2, 1), (3, 1))

Nodes explored: 892, Time taken: 0.0125s

Board after move:

```

- - - - -
B _ _ W _ _
B _ W W B B

```

```

W W W B W B
B W B W B W
-----
Round 12: Player B thinking...
Available moves: [((2, 0), (3, 0)), ((2, 4), (3, 4)), ((2, 4), (2, 3)), ((3, 3), (4, 3)), ((3, 3), (2, 3)), ((3, 3), (3, 4)), ((3, 3), (3, 2)), ((3, 5), (4, 5)), ((3, 5), (3, 4)), ((4, 0), (3, 0)), ((4, 0), (4, 1)), ((4, 2), (3, 2)), ((4, 2), (4, 3)), ((4, 2), (4, 1)), ((4, 4), (3, 4)), ((4, 4), (4, 5)), ((4, 4), (4, 3))]
Player B chose move: ((2, 0), (3, 0))
Nodes explored: 572, Time taken: 0.0080s
Board after move:
- - - - -
B _ _ W _ _
- _ W W B B
B W W B W B
B W B W B W
-----
Round 13: Player W thinking...
Available moves: [((2, 3), (3, 3)), ((2, 3), (2, 4)), ((3, 1), (3, 0)), ((3, 2), (4, 2)), ((3, 2), (3, 3)), ((3, 4), (4, 4)), ((3, 4), (2, 4)), ((3, 4), (3, 5)), ((3, 4), (3, 3)), ((4, 1), (4, 2)), ((4, 1), (4, 0)), ((4, 3), (3, 3)), ((4, 3), (4, 4)), ((4, 3), (4, 2)), ((4, 5), (3, 5)), ((4, 5), (4, 4))]
Player W chose move: ((2, 3), (3, 3))
Nodes explored: 443, Time taken: 0.0057s
Board after move:
- - - - -
B _ _ W _ _
- _ W _ B B
B W W W W B
B W B W B W
-----
Round 14: Player B thinking...
Available moves: [((2, 4), (3, 4)), ((3, 0), (3, 1)), ((3, 5), (4, 5)), ((3, 5), (3, 4)), ((4, 0), (4, 1)), ((4, 2), (3, 2)), ((4, 2), (4, 3)), ((4, 2), (4, 1)), ((4, 4), (3, 4)), ((4, 4), (4, 5)), ((4, 4), (4, 3))]
Player B chose move: ((2, 4), (3, 4))
Nodes explored: 279, Time taken: 0.0035s
Board after move:
- - - - -
B _ _ W _ _
- _ W _ _ B
B W W W B B
B W B W B W
-----
Round 15: Player W thinking...

```

Available moves: [((3, 1), (3, 0)), ((3, 2), (4, 2)), ((3, 3), (3, 4)), ((4, 1), (4, 2)), ((4, 1), (4, 0)), ((4, 3), (4, 4)), ((4, 3), (4, 2)), ((4, 5), (3, 5)), ((4, 5), (4, 4))]

Player W chose move: ((3, 1), (3, 0))

Nodes explored: 214, Time taken: 0.0024s

Board after move:

```

- - - - -
B _ _ W _ _
- _ W _ _ B
W _ W W B B
B W B W B W

```

Round 16: Player B thinking...

Available moves: [((3, 4), (3, 3)), ((3, 5), (4, 5)), ((4, 0), (3, 0)), ((4, 0), (4, 1)), ((4, 2), (3, 2)), ((4, 2), (4, 3)), ((4, 2), (4, 1)), ((4, 4), (4, 5)), ((4, 4), (4, 3))]

Player B chose move: ((3, 4), (3, 3))

Nodes explored: 177, Time taken: 0.0019s

Board after move:

```

- - - - -
B _ _ W _ _
- _ W _ _ B
W _ W B _ B
B W B W B W

```

Round 17: Player W thinking...

Available moves: [((3, 0), (4, 0)), ((3, 2), (4, 2)), ((3, 2), (3, 3)), ((4, 1), (4, 2)), ((4, 1), (4, 0)), ((4, 3), (3, 3)), ((4, 3), (4, 4)), ((4, 3), (4, 2)), ((4, 5), (3, 5)), ((4, 5), (4, 4))]

Player W chose move: ((3, 0), (4, 0))

Nodes explored: 158, Time taken: 0.0021s

Board after move:

```

- - - - -
B _ _ W _ _
- _ W _ _ B
- _ W B _ B
W W B W B W

```

Round 18: Player B thinking...

Available moves: [((3, 3), (4, 3)), ((3, 3), (3, 2)), ((3, 5), (4, 5)), ((4, 2), (3, 2)), ((4, 2), (4, 3)), ((4, 2), (4, 1)), ((4, 4), (4, 5)), ((4, 4), (4, 3))]

Player B chose move: ((3, 3), (4, 3))

Nodes explored: 80, Time taken: 0.0011s

Board after move:

```

- - - - -
B _ _ W _ _

```

```

- - W - - B
- - W - - B
W W B B B W
-----

Round 19: Player W thinking...
Available moves: [((3, 2), (4, 2)), ((4, 1), (4, 2)), ((4, 5), (3, 5)), ((4, 5), (4, 4))]
Player W chose move: ((4, 5), (3, 5))
Nodes explored: 39, Time taken: 0.0004s
Board after move:

- - - - -
B - - W - -
- - W - - B
- - W - - W
W W B B B -
-----

Round 20: Player B thinking...
Available moves: [((2, 5), (3, 5)), ((4, 2), (3, 2)), ((4, 2), (4, 1))]
Player B chose move: ((4, 2), (3, 2))
Nodes explored: 16, Time taken: 0.0002s
Board after move:

- - - - -
B - - W - -
- - W - - B
- - B - - W
W W - B B -
-----

Round 21: Player W thinking...
Available moves: [((2, 2), (3, 2)), ((3, 5), (2, 5))]
Player W chose move: ((2, 2), (3, 2))
Nodes explored: 5, Time taken: 0.0001s
Board after move:

- - - - -
B - - W - -
- - - - - B
- - W - - W
W W - B B -
-----

Round 22: Player B thinking...
Available moves: [((2, 5), (3, 5))]
Player B chose move: ((2, 5), (3, 5))
Nodes explored: 2, Time taken: 0.0000s
Board after move:

- - - - -
B - - W - -
- - - - -
- - W - - B

```

```

W W _ B B _
-----
Player W has no more moves. Game over.
- - - - -
B _ _ W _ _
- - - - -
_ _ W _ _ B
W W _ B B _
22 rounds, winner W
33058
0.6228

```

1.5.3 Przykład 3: Porównanie Wydajności

Ten przykład porównuje wydajność (przebadane węzły i czas wykonania) między algorytmami minimax i alpha-beta pruning na tej samej głębokości.

```

[ ]: # python main.py --heur1 count --heur2 count --depth 4 --method minimax
- - - - -
_ _ B _ _ W
_ B _ _ W _
B _ B _ _ W
B B _ W W W
18 rounds, winner B
9440461
27.9712

# python main.py --heur1 count --heur2 count --depth 4 --method alphabeta
- - - - -
_ _ B _ _ W
_ B _ _ W _
B _ B _ _ W
B B _ W W W
18 rounds, winner B
32814
0.2806

```

1.6 5. Analiza i Obserwacje

1.6.1 5.1. Efektywność Algorytmów

Porównanie algorytmów minimax i alpha-beta pruning na tej samej głębokości (przykład 3) pokazuje dramatyczną różnicę w wydajności:

Algorytm	Przeszukane Węzły	Czas Wykonania (s)	Przyspieszenie
Minimax	9 440 461	27.9712	1x
Alpha-Beta	32 814	0.2806	~99.7x

Widoczne jest, że: - Alpha-beta pruning przeszukał około **288 razy mniej węzłów** niż minimax - Alpha-beta pruning był **prawie 100 razy szybszy** w czasie wykonania - Oba algorytmy doprowadziły do tego samego wyniku gry (18 rund, zwycięzca B)

To potwierdza teoretyczną przewagę algorytmu alpha-beta, który poprzez inteligentne przycinanie drzewa gry znacznie ogranicza przestrzeń poszukiwań bez wpływu na jakość podejmowanych decyzji. Natomiast, należałoby przeprowadzić setki tysięcy takich gier w celu udowodnienia tego metodami statystycznymi.

1.6.2 5.2. Skuteczność Heurystyk

Przykład 1 pokazuje grę, w której gracze korzystają z różnych funkcji heurystycznych: - Gracz B (czarny) używał heurystyki **count** (opartej na liczbie pionków) - Gracz W (biały) używał heurystyki **mobility** (opartej na liczbie dostępnych ruchów)

Wynik: **18 rund, zwycięzca B**

W tym przypadku heurystyka oparta na liczbie pionków okazała się skuteczniejsza. Może to sugerować, że w grze Clobber, w szczególności na wczesnych i średnich etapach gry, ważniejsze jest utrzymanie przewagi materialnej (więcej pionków) niż mobilności. Ponownie, należałoby przeprowadzić setki tysięcy takich gier w celu udowodnienia tego metodami statystycznymi.

1.6.3 5.3. Obserwacje z Przykładu 2

W przykładzie 2 obaj gracze używali tej samej heurystyki **combo** (połączenie liczby pionków i mobilności), ale z grą rozpoczynał gracz W (biały):

Co można zauważyć: - Gra trwała **22 rundy** (dłużej niż w innych przykładach) - Zwyciężył gracz W (biały) - Przeszukano łącznie **33 058 węzłów**

Widoczne jest, że liczba przeszukiwanych węzłów systematycznie malała w trakcie gry: - Runda 1: 5442 węzłów - Runda 10: 1269 węzłów - Runda 20: 55 węzłów

Jest to oczekiwane zachowanie, ponieważ wraz z usuwaniem pionków z planszy zmniejsza się liczba możliwych ruchów, a tym samym rozmiar drzewa gry.

1.6.4 5.4. Wnioski dotyczące głębokości przeszukiwania

Porównując przykłady 1 i 3 (minimax): - Głębokość 3: **364 222** węzłów, **3.48** sekundy - Głębokość 4: **9 440 461** węzłów, **27.97** sekund

Zwiększenie głębokości przeszukiwania o 1 poziom spowodowało: - Około **26-krotny wzrost** liczby przeszukiwanych węzłów - Około **8-krotny wzrost** czasu wykonania

To potwierdza wykładniczą złożoność algorytmów przeszukiwania drzewa gry i pokazuje, jak ważna jest optymalizacja (taka jak alpha-beta pruning) przy głębszych przeszukiwaniach.

1.7 6. Wnioski

1. Przycinanie alpha-beta znacznie redukuje przestrzeń wyszukiwania w porównaniu z minimax, znajdując te same optymalne ruchy.
2. Wybór funkcji heurystycznej znacząco wpływa na zachowanie i wydajność agenta.

3. Głębokość wyszukiwania stanowi kompromis między jakością decyzji a kosztem obliczeniowym.
4. Połączenie liczby pionków i mobilności (heurystyka combo) generalnie zapewnia bardziej zrównoważoną ocenę niż każda z tych miar osobno.

1.8 7. Spełnienie punktu 5 z instrukcji

Punkt 5 z instrukcji wymaga:

Modyfikacji programu do wersji rozszerzonej, co umożliwi rozegranie partii pomiędzy dwoma programami (np. dwoma wywołaniami tej samej aplikacji) oraz zastosowanie heurystyk adaptacyjnie zmieniających strategię gracza w celu osiągnięcia zwycięstwa (dodatkowe 20 punktów).

Analizując obecną implementację projektu, można stwierdzić, że:

1. **Rozgrywka między dwoma programami:** Ten warunek jest spełniony, ponieważ nasz program umożliwia rozegranie partii między dwoma agentami AI (funkcja `play` w `main.py`). Jednak nie zaimplementowano mechanizmu komunikacji między dwoma oddzielnymi wywołaniami aplikacji, który został zaproponowany. Uważa się, że na potrzeby projektu ta implementacja była lepszym wyborem z racji na mniejszy nakład pracy. Wprowadzenie komunikacji między dwoma agentami wymagałoby niepotrzebnego wyjścia poza sam kurs Sztucznej Inteligencji.
2. **Adaptacyjne heurystyki:** Obecna implementacja nie zawiera heurystyk adaptacyjnie zmieniających strategię gracza w trakcie rozgrywki. Dostępne heurystyki (`count`, `mobility`, `combo`) są statyczne i nie dostosowują się do bieżącego stanu gry ani nie zmieniają swojej strategii w trakcie rozgrywki.

W obecnej wersji projektu dostępny jest parametr `--mode` z opcjami `basic` i `ext` w interfejsie wiersza poleceń (linia 69 w `main.py`).

Aby spełnić ten dodatkowy punkt należałoby rozszerzyć klasę `Agent` o możliwość adaptacyjnego wyboru heurystyk w zależności od bieżącej sytuacji na planszy.

Podsumowując, obecna implementacja spełnia ** w większości** dodatkowy punkt 5 z instrukcji.

1.9 8. Naprawy błędów

1.9.1 8.1 Błąd w rekurencyjnych wywołaniach algorytmów

W oryginalnej implementacji algorytmów minimax i alpha-beta występował błąd związany z obsługą wartości zwracanych przez funkcje rekurencyjne owinięte w dekorator `Timer`. Błąd powodował następujące wyjątki:

```
TypeError: '<' not supported between instances of 'tuple' and 'float'  
TypeError: '>' not supported between instances of 'tuple' and 'float'
```

Problem wynikał z tego, że funkcja rekurencyjna owinięta w dekorator `Timer` zwracała krotkę (wynik, czas), ale w kodzie próbowaliśmy bezpośrednio porównywać tę krotkę z wartościami zmiennoprzecinkowymi.

Rozwiązanie Po kilku próbach naprawienia problemu, zastosowaliśmy bardziej gruntowne rozwiązanie, polegające na reorganizacji kodu:

1. Oddzieliliśmy logikę algorytmów od pomiaru czasu, tworząc “funkcje-rdzenie” bez dekoratora Timer, które są używane do rekurencyjnych wywołań.
2. Zastosowaliśmy dekorator Timer tylko do funkcji opakowujących, które wywołują pierwszy poziom rekurencji.

W pliku `minimax.py`:

```
# Przed zmianą
@Timer
def _minimax(node, pl, d):
    # rekurencyjne wywołanie _minimax

# Po poprawce
# Funkcja bez dekoratora dla rekurencyjnych wywołań
def _minimax_core(node, pl, d):
    # kod algorytmu z bezpośrednimi wywołaniami _minimax_core

# Funkcja z dekoratorem tylko dla pierwszego wywołania
@Timer
def _minimax_timed():
    return _minimax_core(board, player, depth)
```

Podobną zmianę zastosowano w pliku `alphabeta.py`. Dzięki temu rekurencyjne wywołania nie są już opakowane w dekorator Timer, co eliminuje problem z porównywaniem krotek z liczbami zmiennoprzecinkowymi.

Po wprowadzeniu tych poprawek, algorytmy działają prawidłowo i można przeprowadzić testy zgodnie z opisem w rozdziale 4.

1.10 9. Dodatkowe problemy implementacyjne

Podczas pracy nad projektem napotkaliśmy kilka dodatkowych wyzwań, które warto odnotować:

1. **Różne warianty gry Clobber:** Gra Clobber występuje w wielu wariantach i wersjach, różniących się rozmiarem planszy, początkowym ułożeniem pionków czy szczegółowymi zasadami. Nasza implementacja jest dostosowana do konkretnej wersji opisanej w treści zadania, ale może wymagać modyfikacji przy adaptacji do innych wariantów gry.
2. **Ograniczenia głębokości przeszukiwania:** Przy większych planszach lub złożonych pozycjach nawet algorytm alpha-beta może napotykać ograniczenia wydajnościowe. Szczególnie dla głębokości przeszukiwania powyżej 4-5 poziomów, czas obliczeń może drastycznie wzrosnąć.
3. **Reprezentacja planszy:** Przyjęta reprezentacja planszy jako dwuwymiarowej tablicy znaków jest prosta w implementacji, ale może nie być optymalna pod względem wydajności. Bardziej zaawansowane reprezentacje (np. bitboardy) mogłyby przyspieszyć generowanie ruchów i ocenę pozycji.

4. **Dobieranie heurystyk:** Dobór odpowiednich funkcji oceniających pozycję na planszy stanowił wyzwanie. Implementowane heurystyki są dość podstawowe i mogą nie uwzględniać wszystkich niuansów strategii w grze Clobber.
5. **Problemy z dekoratorem Timer:** Jak wspomniano wcześniej, zastosowanie dekoratora do mierzenia czasu w funkcjach rekurencyjnych spowodowało nieoczekiwane błędy w porównywaniu zwracanych wartości.

Warto zaznaczyć, że rozwiązanie prezentowane w tym raporcie koncentruje się na implementacji algorytmów przeszukiwania drzewa gry, a nie na optymalizacji konkretnej strategii dla gry Clobber. W przyszłych wersjach można by rozważyć implementację bardziej wyrafinowanych heurystyk specyficznych dla różnych wariantów tej gry.